Transactions Are Events

Event-Driven Architecture On Data In Motion

Matt Gordon
Director of Data and Analytics, Lean TECHniques

Matt Gordon





Director of Data and Analytics

Lean TECHniques

mgordon@leantechniques.com

@sqlatspeed

https://www.sqlatspeed.com

Matt is a Microsoft Data Platform MVP and Redgate Community Ambassador with 20+ years of data experience and nearly 10 years of Azure AI experience.

He currently co-organizes the Lexington (KY) Data Technology Group.

The Big Idea

"A committed transaction isn't just a row change. It's an event - and now your database can publish it."

- COMMIT *can be* a business moment in time
- Change Event Streaming (CES) lets Azure SQL DB / SQL Server 2025 push that moment out as an event
- Two concepts to ponder during the session:
  - Change Event Streaming - how the database publishes committed transactions as events
  - Action Systems - what we build to react to those events

Agenda

- Rethinking transactions as events
- Data in motion vs. data at rest
- Change Event Streaming (CES) - what is it and what can it do for me?
- How CES differs from CDC
- Streaming into Microsoft Fabric with Eventstream
- Action Systems - what they are and how we all have them
- Demo Time!
- What if I don’t have Fabric?
- How can this help me rethink my ETL?
- Takeaways and Resources

The ~~Old~~ Current Model - Data at Rest

- ETL on a schedule (nightly batch)
- Reports tell you what happened before
- Applications and pipelines poll for changes, orchestration gets very intricate as this scales
- Insight likely isn’t current, but may need to be current (or even predictive)
- What have we missed and what gaps have we left as we’re waiting for the next batch execution?

The New/Upcoming Model - Data in Motion

- The database emits change the instant a transaction commits
- We all have this kind of data in motion
- Systems can react rather than just record
- Analytics run continuously, not on a schedule
- Decisions can shift from “what happened?” to “what's happening?” to “what should we do about it?”

What Is Event-Driven Architecture?

- Producers/sources emit events; consumers/targets can receive and react to those events
- End-to-end flow: event source → event stream → event processor → action
- Historically used to require 3rd-party tools or internal, bespoke development and maintenance
- CES turns the SQL Server/Azure SQL database engine into a native event producer

Change Event Streaming – What Is It and What Does It Do?

- Change Event Streaming (CES) is a native feature of Azure SQL Database and SQL Server 2025
- Pushes committed transactions as events to an external destination
- Supported destinations: Azure Event Hub or Microsoft Fabric Eventstream
- Push-based and near real-time means no polling and no change tables
- This is what makes us able to talk about how “transactions are events”

CES vs. CDC - Two Completely Different Features

	Change Event Streaming (CES)	Change Data Capture (CDC)
Model	Push - emits events	Pull - populates change tables
Unit	Committed transaction as an event	Row-level change record
Destination	Event Hub / Fabric Eventstream (external)	Change tables (inside the database)
Consumption	Event-driven, real-time	Query/poll on your schedule
Purpose	Event-driven architecture	Tracking changes / capturing history

They are separate features. One does not feed the other. CES does not require CDC.

Why Do I Think Microsoft Built This?

- CES being native in Azure SQL Database and SQL Server 2025 means no external CDC/log-tailing tooling required

- Microsoft Fabric provides a native destination via Eventstream + Real-Time Intelligence

- Data teams comprised of DBAs and data engineers can work together in this kind of architecture

- Lower barrier to entry for event-driven architecture on relational data (which can also feed AI agents)

- Given how much Microsoft, Databricks, and industry leadership talks about it, if we're not already being asked to make data available to agents, we *will* be asked

The Source - Azure SQL Database & SQL Server 2025

- Still the transactional backbone for countless apps and Microsoft’s most ubiquitous data product
- Every commit can now be a durable, ordered business event with can make ETL/ELT/etc. more resilient
- CES exposes those committed transactions directly as an event stream
- No middleware and no polling loop means the engine does the publishing and there is less to monitor

How CES Works (At A High Level)

- You configure a change event stream on the source database
- Point it at a destination (Event Hub or Fabric Eventstream)
- On commit, the engine emits an event describing the transaction
- Consumers subscribe downstream and react, notify, etc. in near real time

Microsoft Fabric – What Is It?

- SaaS analytics platform unifying:
  - Data Engineering (Spark), Data Factory, Data Warehouse
  - Databases in Fabric (SQL, Cosmos DB)
  - Real-Time Intelligence
  - Power BI

- All data lands on OneLake - one copy, many engines

- Our CES destination lives in the Real-Time Intelligence (RTI) part of Fabric

Real-Time Intelligence in Fabric

- Purpose-built experience for streaming/event data

- Key components:
  - Eventstream - ingest and route events (our CES destination)
  - Eventhouse / KQL Database - store and query at high velocity using the Kusto engine
  - Real-time dashboard – NOT Power BI, but quick data visualization and visual trend analysis
  - Activator - detect conditions and trigger actions (our Action System engine)

Fabric Eventstream - The (Easier) CES Destination

- No-code/low-code event ingestion and routing canvas
- Receives the event stream pushed directly by CES from Azure SQL DB / SQL Server 2025
- Destinations from Eventstream: Eventhouse, Lakehouse, Data Activator, custom endpoints
- In-flight, lightweight transformations: filter, aggregate, expand, manage fields

The CES → Eventstream Path

- CES pushes committed-transaction events out of the database and towards the receiving architecture
- Fabric Eventstream receives them as they commit - no polling, no batch, no schedule, no gaps
- This is a genuine event-driven pipeline, not an ETL job needing to be monitored and repaired
- Once in Fabric, events can fan out to storage, analytics, and action systems

Eventhouse & KQL Database

- Purpose-built for high-volume, time-series/event data
- Kusto Query Language (KQL) - fast, expressive, built for time-windowed analysis
- Ideal sink for CES events for historical + near-real-time analysis
- Powers dashboards that update in seconds, not overnight

What Do I Mean By “Action Systems“?

- Streaming and analyzing events is great, but doesn’t bring us the ability to take quick action if needed
- Action Systems: architectures designed to automatically respond to events, not just report on them
- Concept originally championed in the data community by Christopher Schmidt
- Leaders are increasing looking for insights, automated action, and agents operating on analytics
- The shift: system of record → system of insight → system of action

What Makes a System an “Action System”?

- It can detect a condition in a live stream of events (and a static “report” if needed, but not preferable)
- Evaluates that condition against defined thresholds/rules in near real time
- Triggers an automated response and action - notification, workflow, orchestration, downstream API call
- Closes the loop: event to insight to action with minimal human latency (but critical human oversight)

Activator - Building an Action System in Fabric

- Rules-based, no-code trigger engine – it is Fabric's native Action System component
- "Watches" Eventstreams, KQL data, Real-time Dashboards, or Power BI datasets for conditions
- Fires actions: alerts (Teams/Email), Power Automate flows, Fabric artifact initiation, webhook calls
- CES can deliver the event but Activator turns it into an action after a human has defined the condition(s)

Today’s Example Architecture

- Azure SQL Database / SQL Server 2025 (Change Event Streaming)
- Fabric Eventstream (receives pushed transaction events)
- Eventhouse/KQL DB (store & analyze)
- Data Activator (Action System layer)
- Real-time Dashboards, Power BI / Teams / Power Automate / Fabric artifacts (visualize & respond)

Demo Overview

- Transactional data flowing into Fabric
- Are there data issues (do you have any issues with data in motion that need monitoring?)
- Visualizing potential data quality or bad actor-induced issues
- How can we take action on these potential issues?
- Did anybody watch soccer this summer…?

Demo - Step 1: Configure CES

- Enable/configure Change Event Streaming on the source database
- Set the destination to a Fabric Eventstream
- Commit some transactions to watch the events flow

Demo - Step 2: Receive in Eventstream

- Open the Fabric Eventstream receiving the CES events
- Watch committed transactions arrive as events in near real time
- Route to Eventhouse (KQL DB)

Demo - Step 3: Action Systems with Activator

- Illustrate how to define a Data Activator trigger condition on the streamed field
- What actions can we configure?
- What type of actions do you have (or do you need) in your environment?

What If I Have Databricks Instead of Fabric?

- To receive CloudEvents in Databricks, you could use Databricks Jobs with File Arrival Triggers or Auto Loader with Cloud Storage Notifications

- Eventstream equivalent: Structured Streaming / Delta Live Tables (Lakeflow Declarative Pipelines)

- Action Systems in Databricks are typically hand-built (Azure Functions, Logic Apps, workflow jobs) as there is no true Activator equivalent

- Tale of the tape: Fabric is easier here (but that doesn’t mean it’s best for you overall)

Fabric vs. Databricks – Which Horse for Which Course?

Consideration	Microsoft Fabric	Databricks
Team skillset	SQL / Power BI / low-code	Spark / Python / notebooks
Native CES destination	Eventstream (built-in)	Requires custom ingestion
Native Action System	Data Activator (no-code)	Custom-built (Functions/Logic Apps)
Governance	Microsoft Purview / OneLake	Unity Catalog

Not mutually exclusive - many shops use both. This talk stays Microsoft-first because that's where most transactional estates already live.

Design Patterns & Pitfalls

- **Patterns:**
  - Scope CES to the tables/transactions that genuinely need real-time delivery
  - Keep Eventstream transformations light and push heavy logic farther downstream
  - Design Action System rules to be specific - alert fatigue always kills adoption

- **Pitfalls:**
  - Confusing CES with CDC - they solve different problems and are not truly equivalent
  - Over-triggering the Action System means everybody will ignore and overlook the cool thing you just built
  - Assuming “real-time” means zero latency - define your actual SLA and communicate this repeatedly

How Can This Help Me Rethink ETL – and Should It?

"If transactions are events, what happens to ETL?"

- Everything we've built so far isn't a new pipeline - it's a different methodology for data movement
- Event-driven architecture should drive analysis on where you do (and do not) need traditional ETL
- How can CES and Eventstream reshape our thinking around data we put in motion to support analytics?

Tried and True Batch-Style ETL

- Extract on a schedule (nightly, hourly, etc.)
- Poll the source: “what changed since last run?”
- Stage it, transform it, load in a fixed window on a fixed schedule
- Assumes that latency is acceptable, the batch window exists, and the source can absorb the query load
- Can we question those assumptions?

Should We Be “Subscribing” Instead Of Extracting?

- Traditional ETL: the pipeline reaches into the source and pulls data out
- Event-driven: the source (via CES) pushes committed transactions out and the pipeline "subscribes“
- We’ve always asked, “what changed since last run?” but now the database tells us at commit time
- Complex extract logic (change detection, watermarks, delta queries) is much less necessary

Batch Windows Can Disappear

- No more “the load must finish by 6 AM” fire drills
- Processing is continuous, not a nightly event
- Data freshness goes from hours/days to seconds
- Freshness becomes a design choice, not a constraint imposed by the schedule
- Fewer late-night alerts when a batch job runs long or fails

Incremental by Default

- Batch ETL requires us to bolt on “incremental logic” even if the data doesn’t easily support it
- With CES, every event is already the increment - one committed transaction, one event
- No high-water-mark tables, no “did we miss rows?” reconciliation anxiety (and the complex repairs that come from that)
- Far less custom plumbing to build, test, and maintain

Lighter Load on the Source System

- Polling-based ETL repeatedly and regularly queries production to find changes - competing with the OLTP workload but with two very different workload shapes
- CES emits events as a byproduct of commits, rather than being repeatedly interrogated
- The reporting/analytics tier stops taxing the transactional tier on a schedule
- Better for the DBA and the analytics team – servers without performance spikes are happy servers!

Transform In-Flight, Not Just At-Rest

- Batch forces us to land raw data and then transform in staging (or later)

- Event-driven allows us to filter, enrich, aggregate, and shape events in the stream (Eventstream transformations) - and continue refining downstream

- “ETL vs. ELT” gives way to streaming transformation: transform where it makes sense, when the event arrives

- Heavy/complex logic can still (and will) live downstream (KQL, Lakehouse, Spark) - but we’re doing far less work to send the data in motion and maintaining far fewer representations of the data

The Medallion Architecture In Motion

- Bronze/Silver/Gold (or Raw/Enriched/Curated) doesn't require batch
- Bronze/Raw: CES events land in Eventhouse/Lakehouse as they arrive
- Silver/Enriched: cleansed/conformed continuously via in-flight streaming transforms
- Gold/Curated: aggregated and serving real-time dashboards, Action Systems, and reports
- Data governance doesn’t change (and lineage is easier to trace)

New Stuff You Now Have to Think About

- Ordering & delivery: events are per-transaction so we need to design consumers accordingly
- Idempotency: how do we handle reprocessing if an issue upstream requires it?
- Retention: where is the ultimate destination of this data (which typically serves reports)?
- Schema evolution: source schema changes ripple into the stream - plan for drift
- Monitoring changes from “did the job finish?” to “is the stream healthy and flowing?”

Batch ETL Isn't Dead – But We Have Options

- Event-driven ETL is a powerful default for fresh, incremental workloads
- Batch still wins for large historical backfills, heavy multi-source joins, periodic full reconciliations, and workloads with no freshness requirement
- Truth is in the middle (maybe): CES/Eventstream for data in motion, batch for the bulk/periodic jobs
- I’m not saying “never batch ETL again” but I am saying “it’s time to stop batching things that no longer need to be batched”

Key Takeaways

1. Every committed transaction can be an event and CES can push it natively from Azure SQL DB / SQL Server 2025

2. CES and CDC are different features - push vs. pull, event delivery vs. change tables - one does not feed the other but they can be complementary in certain architectures

3. Fabric Eventstream and Azure Event Hubs are native CES destinations in the Microsoft ecosystem

4. Activator makes building an Action System accessible to everyone, not just engineers

Resources

- https://learn.microsoft.com/en-us/sql/relational-databases/track-changes/change-event-streaming/overview?view=sql-server-ver17
- https://learn.microsoft.com/en-us/fabric/real-time-intelligence/event-streams/stream-sql-change-events-to-eventstream
- https://realtimedispatch.substack.com/ - Christopher Schmidt's Real-time Dispatch blog
- https://www.youtube.com/watch?v=2j-y_YtR2r0 – my video of this demo on Data Exposed
- Contact me: @sqlatspeed everywhere online

- **Questions?**